EMIPredict AI - Intelligent Financial Risk Assessment Platform

```
DATA_PATH = "/content/emi_prediction_dataset_cleaned.csv"

print("Dataset path:")
print(DATA_PATH)
```

```
Dataset path:
/content/emi_prediction_dataset_cleaned.csv
```

```
import pandas as pd

df = pd.read_csv(DATA_PATH)

print("Dataset loaded successfully!")
print("Shape:", df.shape)

display(df.head())
```

company_type	house_type	m
Mid-size	Rented	
MNC	Family	
Startup	Own	
Mid-size	Own	
Mid-size	Family	

Install Required Libraries

```
!pip install -q pandas numpy scikit-learn xgboost mlflow gradio plotly joblib openpyxl
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49.7/49.7 kB 630.6 kB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 50.5/50.5 kB 1.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 1.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.2/11.2 MB 46.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.6/3.6 MB 101.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 68.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 265.9/265.9 kB 21.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.7/4.7 MB 67.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 148.8/148.8 kB 11.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 114.9/114.9 kB 7.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 228.4/228.4 kB 15.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 123.9/123.9 kB 9.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 132.2/132.2 kB 10.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.1/1.1 MB 58.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 222.1/222.1 kB 16.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 216.0/216.0 kB 15.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 72.5/72.5 kB 4.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.8/62.8 kB 5.1 MB/s eta 0:00:00
```

Import Libraries

```
import os
import warnings
import joblib
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import plotly.express as px

warnings.filterwarnings("ignore")
```

```
from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler,
    LabelEncoder
)

from sklearn.impute import SimpleImputer

from sklearn.linear_model import (
    LogisticRegression,
    LinearRegression
)

from sklearn.ensemble import (
    RandomForestClassifier,
    RandomForestRegressor
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

from xgboost import (
    XGBClassifier,
    XGBRegressor
)

import mlflow
import mlflow.sklearn

import gradio as gr

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

Load Your Cleaned Dataset

```
df = pd.read_csv(DATA_PATH)

print("Dataset loaded successfully!")
print("=" * 60)

print("Rows    :", df.shape[0])
print("Columns :", df.shape[1])

print("\nColumn Names:")
print(df.columns.tolist())

display(df.head())
```

```
Dataset loaded successfully!
============================================================
Rows    : 404310
Columns : 27

Column Names:
['age', 'gender', 'marital_status', 'education', 'monthly_salary', 'employment_type', 'years_of_employment', 'company_type',
```

	age	gender	marital_status	education	monthly_salary	employment_type	years_of_employment	company_type	house_type
0	38	Female	Married	Professional	82600.0	Private	0.9	Mid-size	Rented
1	38	Female	Married	Graduate	21500.0	Private	7.0	MNC	Family
2	38	Male	Married	Professional	86100.0	Private	5.8	Startup	Own
3	58	Female	Married	High School	66800.0	Private	2.2	Mid-size	Own
4	48	Female	Married	Professional	57300.0	Private	3.4	Mid-size	Family

5 rows × 27 columns

Dataset Validation

```
expected_columns = [
    "age",
    "gender",
    "marital_status",
    "education",
    "monthly_salary",
    "employment_type",
    "years_of_employment",
    "company_type",
    "house_type",
    "monthly_rent",
    "family_size",
    "dependents",
    "school_fees",
    "college_fees",
    "travel_expenses",
    "groceries_utilities",
    "other_monthly_expenses",
    "existing_loans",
    "current_emi_amount",
    "credit_score",
    "bank_balance",
    "emergency_fund",
    "emi_scenario",
    "requested_amount",
    "requested_tenure",
    "emi_eligibility",
    "max_monthly_emi"
]

missing_columns = [
    col for col in expected_columns
    if col not in df.columns
]

if len(missing_columns) == 0:
    print("✅ All expected columns are present.")

else:
    print("❌ Missing columns:")
    print(missing_columns)
```

```
✅ All expected columns are present.
```

Target Distribution

```
print("EMI Eligibility Distribution")
```

```
display(
    df["emi_eligibility"]
    .value_counts()
    .to_frame("Count")
)
```

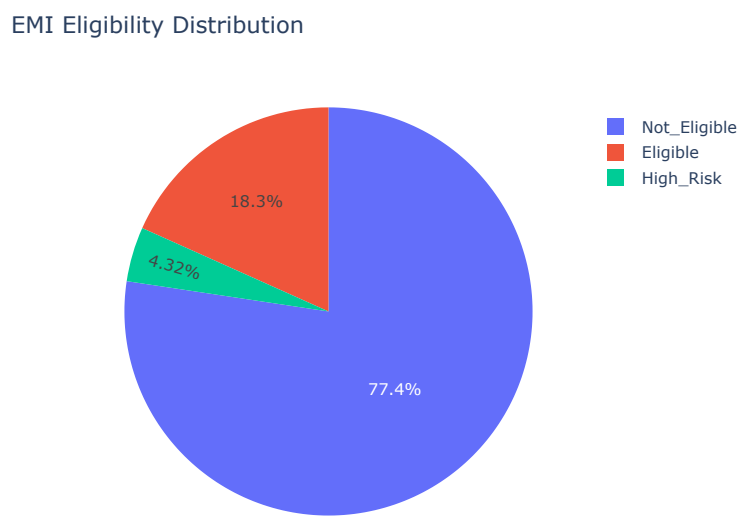
EMI Eligibility Distribution

emi_eligibility	Count
Not_Eligible	312826
Eligible	74000
High_Risk	17484

EDA 1: Credit Score

```
fig = px.pie(
    df,
    names="emi_eligibility",
    title="EMI Eligibility Distribution"
)

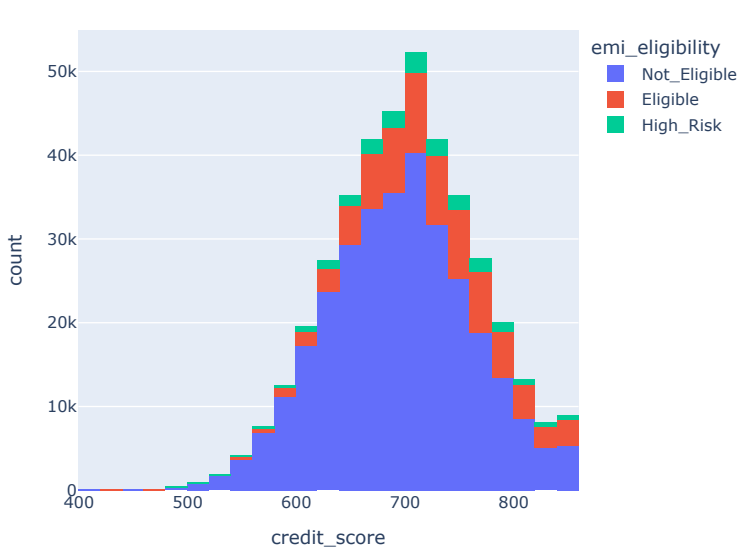
fig.show()
```



EDA 1: Credit Score

```
fig = px.histogram(
    df,
    x="credit_score",
    color="emi_eligibility",
    nbins=40,
    title="Credit Score Distribution by EMI Eligibility"
)

fig.show()
```



EDA 2: Monthly Salary

```
fig = px.histogram(
    df,
    x="monthly_salary",
    color="emi_eligibility",
    nbins=40,
    title="Monthly Salary Distribution"
)

fig.show()
```

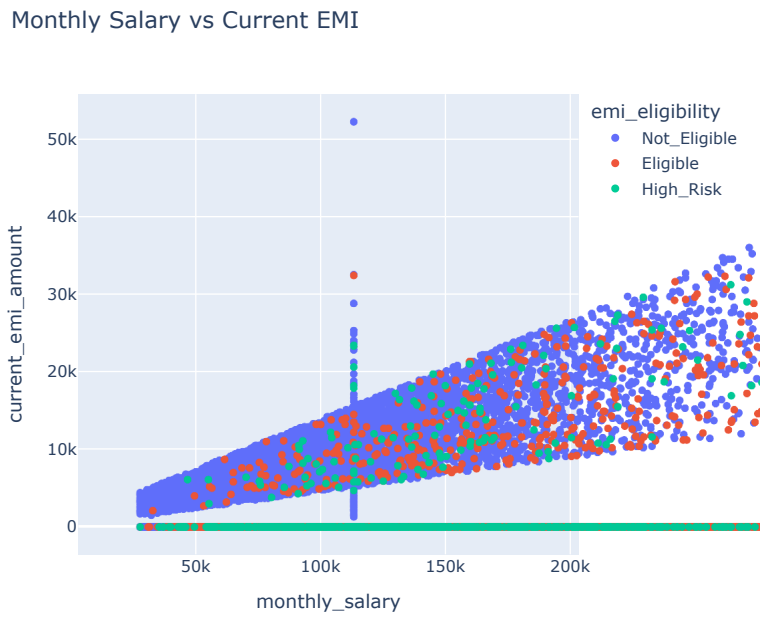


EDA 3: Salary vs Current EMI

```
sample_df = df.sample(
    min(15000, len(df)),
    random_state=42
)
```

```
fig = px.scatter(
    sample_df,
    x="monthly_salary",
    y="current_emi_amount",
    color="emi_eligibility",
    title="Monthly Salary vs Current EMI"
)

fig.show()
```



EDA 4: Salary vs Maximum EMI

```
fig = px.scatter(
    sample_df,
    x="monthly_salary",
    y="max_monthly_emi",
    color="emi_eligibility",
    title="Monthly Salary vs Maximum Safe EMI"
)

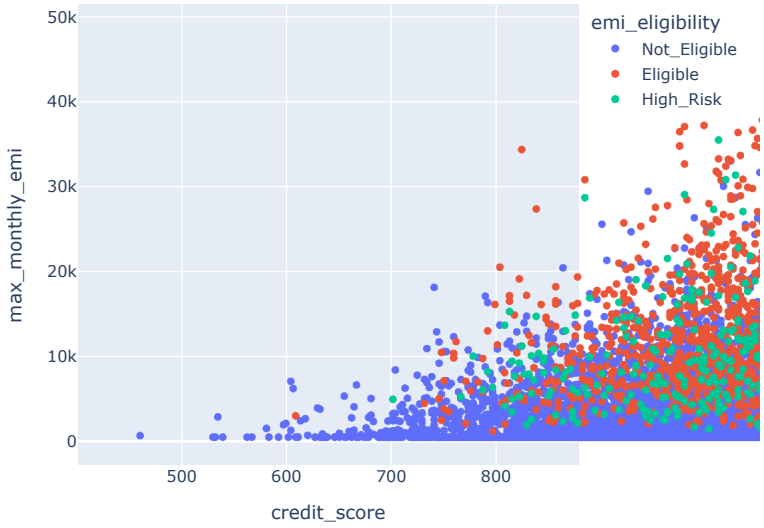
fig.show()
```



EDA 5: Credit Score vs Maximum EMI

```
fig = px.scatter(
    sample_df,
    x="credit_score",
    y="max_monthly_emi",
    color="emi_eligibility",
    title="Credit Score vs Maximum Safe EMI"
)

fig.show()
```



EDA 6: EMI Scenario

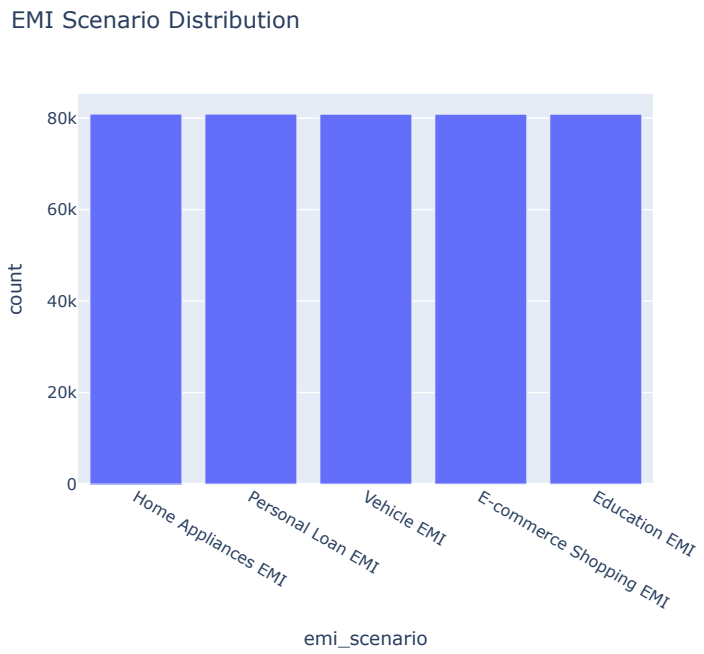
```
scenario_count = (
    df["emi_scenario"]
    .value_counts()
    .reset_index()
```

```
)

scenario_count.columns = [
    "emi_scenario",
    "count"
]

fig = px.bar(
    scenario_count,
    x="emi_scenario",
    y="count",
    title="EMI Scenario Distribution"
)

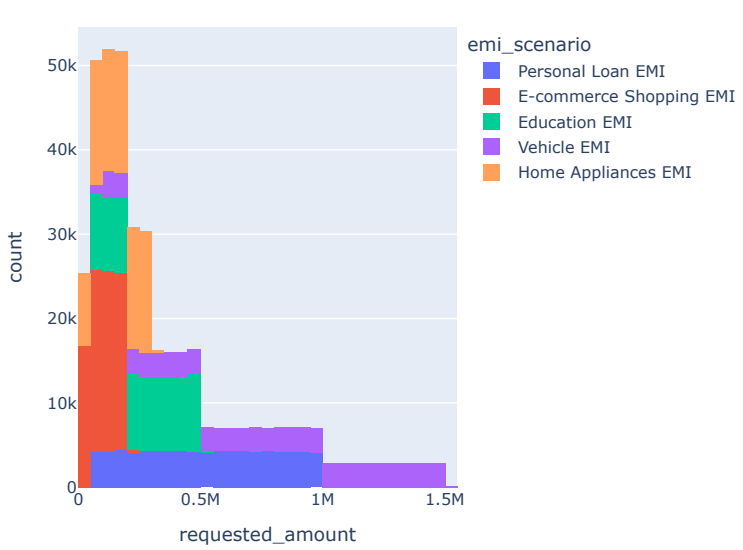
fig.show()
```



EDA 7: Requested Amount

```
fig = px.histogram(
    df,
    x="requested_amount",
    color="emi_scenario",
    nbins=50,
    title="Requested Loan Amount by EMI Scenario"
)

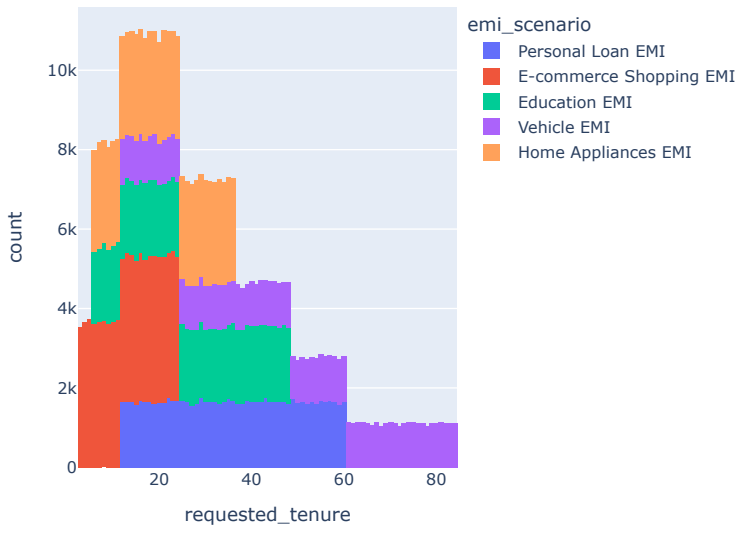
fig.show()
```



EDA 8: Requested Tenure

```
fig = px.histogram(
    df,
    x="requested_tenure",
    color="emi_scenario",
    title="Requested Tenure Distribution"
)

fig.show()
```

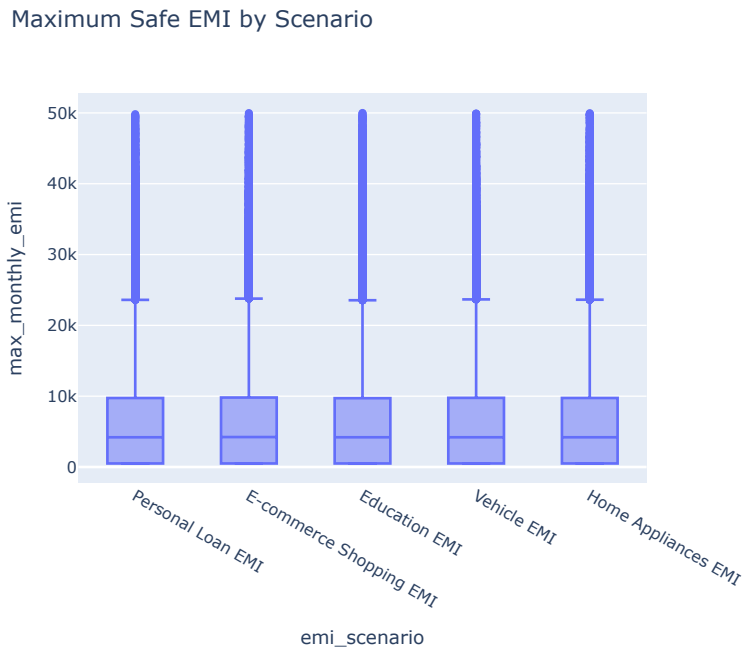


EDA 9: Maximum EMI by Scenario

```
fig = px.box(
    df,
    x="emi_scenario",
    y="max_monthly_emi",
    title="Maximum Safe EMI by Scenario"
```

```
)

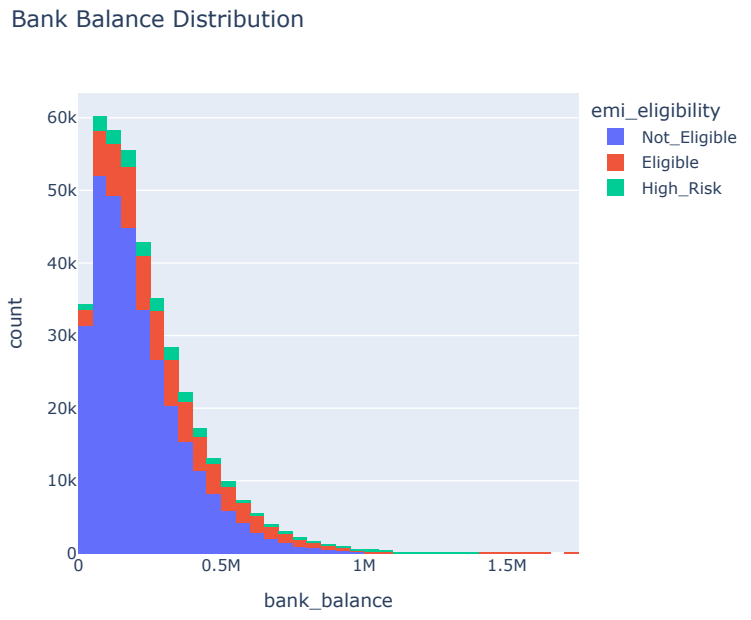
fig.show()
```



EDA 10: Bank Balance

```
fig = px.histogram(
    df,
    x="bank_balance",
    color="emi_eligibility",
    nbins=40,
    title="Bank Balance Distribution"
)

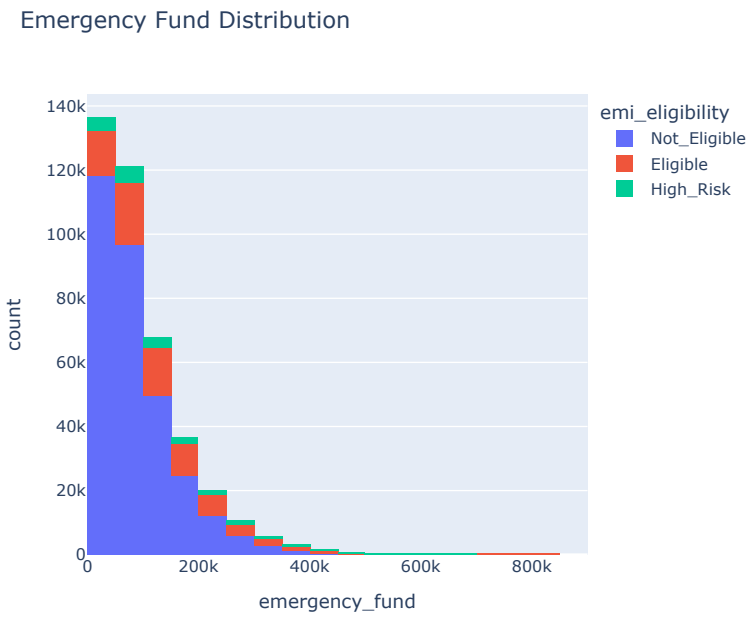
fig.show()
```



EDA 11: Emergency Fund

```
fig = px.histogram(
    df,
    x="emergency_fund",
    color="emi_eligibility",
    nbins=40,
    title="Emergency Fund Distribution"
)

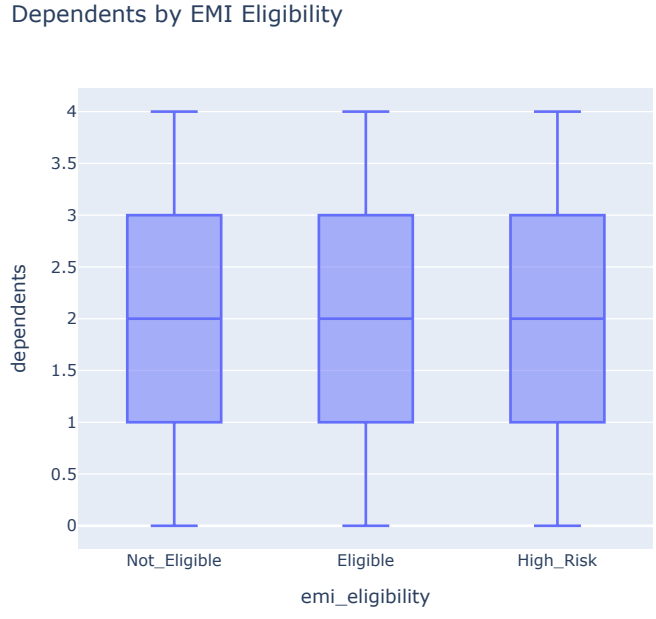
fig.show()
```



EDA 12: Dependents

```
fig = px.box(
    df,
    x="emi_eligibility",
    y="dependents",
    title="Dependents by EMI Eligibility"
)

fig.show()
```



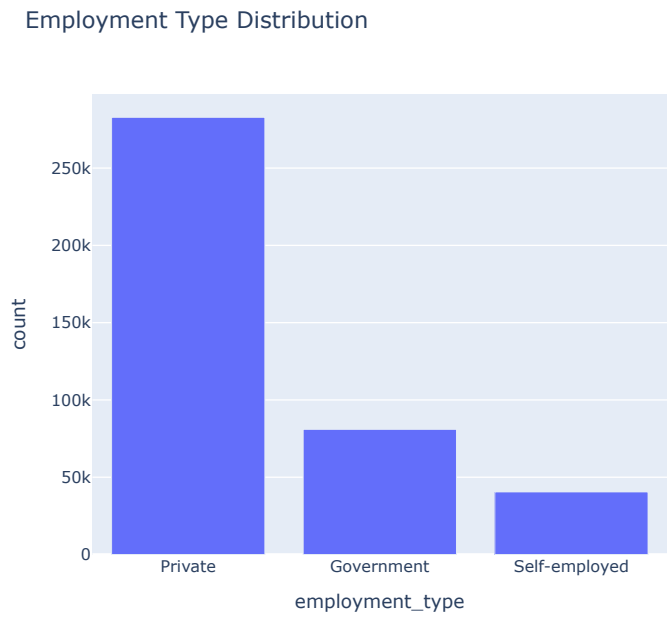
EDA 13: Employment Type

```
employment = (
    df["employment_type"]
    .value_counts()
    .reset_index()
)

employment.columns = [
    "employment_type",
    "count"
]

fig = px.bar(
    employment,
    x="employment_type",
    y="count",
    title="Employment Type Distribution"
)

fig.show()
```



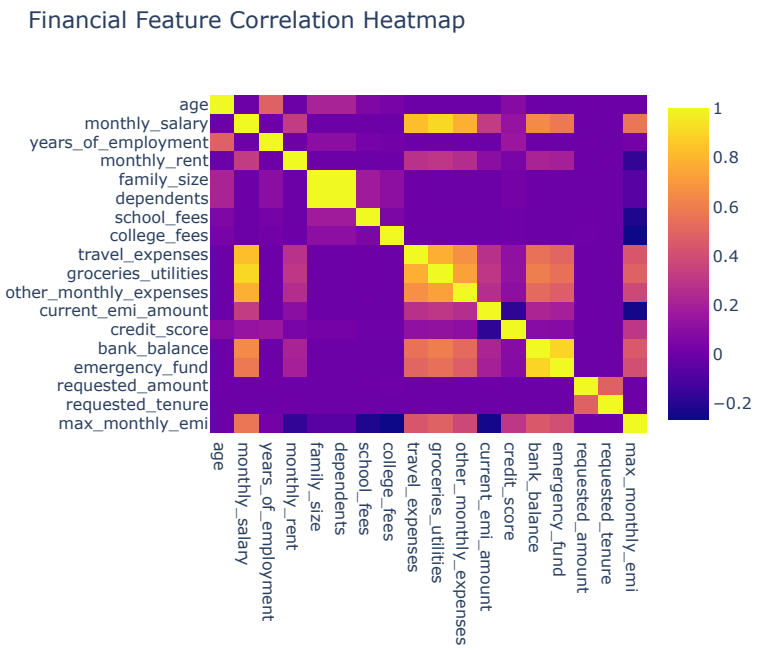
EDA 14: Correlation Heatmap

```
numeric_df = df.select_dtypes(
    include=np.number
)

correlation = numeric_df.corr()

fig = px.imshow(
    correlation,
    title="Financial Feature Correlation Heatmap",
    aspect="auto"
)

fig.show()
```



Feature Engineering

```
def feature_engineering(data):

    data = data.copy()

    salary = data["monthly_salary"].clip(
        lower=1
    )

    # Total monthly expenses
    data["total_monthly_expenses"] = (
        data["monthly_rent"]
        + data["school_fees"]
        + data["college_fees"]
        + data["travel_expenses"]
        + data["groceries_utilities"]
        + data["other_monthly_expenses"]
    )

    # Total financial burden
    data["total_financial_burden"] = (
        data["current_emi_amount"]
        + data["total_monthly_expenses"]
    )

    # Debt to income
    data["debt_to_income"] = (
        data["current_emi_amount"] / salary
    )

    # Expense to income
    data["expense_to_income"] = (
        data["total_monthly_expenses"] / salary
    )

    # Overall burden
    data["total_burden_ratio"] = (
        data["total_financial_burden"] / salary
    )

    # Disposable income
    data["disposable_income"] = (
        salary -
        data["total_financial_burden"]
    )

    # Emergency fund ratio
```

```
    data["emergency_fund_ratio"] = (
        data["emergency_fund"] / salary
    )

    # Bank balance ratio
    data["bank_balance_ratio"] = (
        data["bank_balance"] / salary
    )

    # Requested loan relative to annual income
    data["loan_to_annual_income"] = (
        data["requested_amount"] /
        (salary * 12)
    )

    # Employment stability
    data["employment_stability"] = (
        data["years_of_employment"] /
        data["age"].clip(lower=1)
    )

    # Credit risk indicator
    data["credit_risk_score"] = (
        850 -
        data["credit_score"]
    )

    # Dependency burden
    data["dependent_ratio"] = (
        data["dependents"] /
        data["family_size"].clip(lower=1)
    )

    return data


df_model = feature_engineering(df)

print(
    "Original columns:",
    df.shape[1]
)

print(
    "After feature engineering:",
    df_model.shape[1]
)

display(df_model.head())
```

```
Original columns: 27
After feature engineering: 39
```

	age	gender	marital_status	education	monthly_salary	employment_type	years_of_employment	company_type	house_type
0	38	Female	Married	Professional	82600.0	Private	0.9	Mid-size	Rented
1	38	Female	Married	Graduate	21500.0	Private	7.0	MNC	Family
2	38	Male	Married	Professional	86100.0	Private	5.8	Startup	Own
3	58	Female	Married	High School	66800.0	Private	2.2	Mid-size	Own
4	48	Female	Married	Professional	57300.0	Private	3.4	Mid-size	Family

5 rows × 39 columns

Prepare Targets

```
CLASS_TARGET = "emi_eligibility"
REG_TARGET = "max_monthly_emi"

df_model = df_model.dropna(
    subset=[
        CLASS_TARGET,
        REG_TARGET
    ]
).reset_index(drop=True)
```

```
X = df_model.drop(
    columns=[
        CLASS_TARGET,
        REG_TARGET
    ]
)

y_class_text = (
    df_model[CLASS_TARGET]
    .astype(str)
)

y_reg = (
    df_model[REG_TARGET]
    .astype(float)
)

label_encoder = LabelEncoder()

y_class = label_encoder.fit_transform(
    y_class_text
)

print("Classification classes:")

for i, label in enumerate(
    label_encoder.classes_
):
    print(i, "=", label)

print("\nRegression target:", REG_TARGET)
```

```
Classification classes:
0 = Eligible
1 = High_Risk
2 = Not_Eligible

Regression target: max_monthly_emi
```

Train / Validation / Test Split

```
X_train, X_temp, \
y_train_class, y_temp_class, \
y_train_reg, y_temp_reg = train_test_split(
    X,
    y_class,
    y_reg,
    test_size=0.30,
    random_state=42,
    stratify=y_class
)

X_val, X_test, \
y_val_class, y_test_class, \
y_val_reg, y_test_reg = train_test_split(
    X_temp,
    y_temp_class,
    y_temp_reg,
    test_size=0.50,
    random_state=42,
    stratify=y_temp_class
)

print("Training:", X_train.shape)
print("Validation:", X_val.shape)
print("Testing:", X_test.shape)
```

```
Training: (283017, 37)
Validation: (60646, 37)
Testing: (60647, 37)
```

Preprocessing Pipeline

Preprocessing Pipeline

```
numeric_features = (
    X_train
    .select_dtypes(
        include=[
            "int64",
            "float64",
            "int32",
            "float32"
        ]
    )
    .columns
    .tolist()
)

categorical_features = (
    X_train
    .select_dtypes(
        include=[
            "object",
            "string",
            "category",
            "bool"
        ]
    )
    .columns
    .tolist()
)

numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="median"
        )
    ),
    (
        "scaler",
        StandardScaler()
    )
])

categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    )
])

preprocessor = ColumnTransformer([
    (
        "numeric",
        numeric_pipeline,
        numeric_features
    ),
    (
        "categorical",
        categorical_pipeline,
        categorical_features
    )
])

print(
    "Numerical features:",
    len(numeric_features)
)

print(
```

```
    "Categorical features:",
    len(categorical_features)
)
```

```
Numerical features: 29
Categorical features: 8
```

MLflow Setup

Double-click (or enter) to edit

```
MLFLOW_PATH = (
    "/content/EMIPredict_AI/mlruns"
)

os.makedirs(
    MLFLOW_PATH,
    exist_ok=True
)

mlflow.set_tracking_uri(
    "file://" + MLFLOW_PATH
)

print(
    "MLflow Tracking URI:"
)

print(
    mlflow.get_tracking_uri()
)
```

```
MLflow Tracking URI:
file:///content/EMIPredict_AI/mlruns
```

Train Classification Models

```
classification_models = {

    "Logistic Regression":
        LogisticRegression(
            max_iter=1000,
            class_weight="balanced"
        ),

    "Random Forest":
        RandomForestClassifier(
            n_estimators=120,
            max_depth=18,
            min_samples_split=5,
            class_weight="balanced",
            random_state=42,
            n_jobs=-1
        ),

    "XGBoost":
        XGBClassifier(
            n_estimators=180,
            max_depth=8,
            learning_rate=0.08,
            subsample=0.8,
            colsample_bytree=0.8,
            objective="multi:softprob",
            eval_metric="mlogloss",
            random_state=42,
            n_jobs=-1
        )
}
```

```
os.environ["MLFLOW_ALLOW_FILE_STORE"] = "true"
mlflow.set_experiment(
    "EMIPredict_AI_Classification"
)
```

```
classification_results = []

classification_pipelines = {}

for name, model in classification_models.items():

    print("\n" + "=" * 60)
    print("Training:", name)
    print("=" * 60)

    pipeline = Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            model
        )
    ])

    with mlflow.start_run(
        run_name=name.replace(" ", "_")
    ):

        pipeline.fit(
            X_train,
            y_train_class
        )

        predictions = pipeline.predict(
            X_test
        )

        accuracy = accuracy_score(
            y_test_class,
            predictions
        )

        precision = precision_score(
            y_test_class,
            predictions,
            average="weighted",
            zero_division=0
        )

        recall = recall_score(
            y_test_class,
            predictions,
            average="weighted",
            zero_division=0
        )

        f1 = f1_score(
            y_test_class,
            predictions,
            average="weighted",
            zero_division=0
        )

        try:

            probabilities = (
                pipeline
                .predict_proba(X_test) if hasattr(pipeline, 'predict_proba') else []
            )

            if len(probabilities) > 0:
                roc_auc = roc_auc_score(
                    y_test_class,
                    probabilities,
                    multi_class="ovr",
                    average="weighted"
                )
            else:
                roc_auc = np.nan
```

```
        except:

            roc_auc = np.nan

        mlflow.log_param(
            "model",
            name
        )

        mlflow.log_metric(
            "accuracy",
            accuracy
        )

        mlflow.log_metric(
            "precision",
            precision
        )

        mlflow.log_metric(
            "recall",
            recall
        )

        mlflow.log_metric(
            "f1_score",
            f1
        )

        if not np.isnan(roc_auc):

            mlflow.log_metric(
                "roc_auc",
                roc_auc
            )

        mlflow.sklearn.log_model(
            pipeline,
            name="model",
            skops_trusted_types=['numpy.dtype', 'sklearn.compose._column_transformer._RemainderColsList', 'xgboost
        )

        classification_pipelines[
            name
        ] = pipeline

        classification_results.append({

            "Model": name,

            "Accuracy": accuracy,

            "Precision": precision,

            "Recall": recall,

            "F1 Score": f1,

            "ROC AUC": roc_auc

        })

        print(
            "Accuracy:",
            round(accuracy, 4)
        )

        print(
            "F1:",
            round(f1, 4)
        )
```

```
============================================================
Training: Logistic Regression
============================================================
Accuracy: 0.8326
F1: 0.873
```

```
============================================================
Training: Random Forest
============================================================
Accuracy: 0.9257
F1: 0.9295

============================================================
Training: XGBoost
============================================================
Accuracy: 0.9678
F1: 0.9634
```

```
classification_results_df = pd.DataFrame(
    classification_results
)

classification_results_df = (
    classification_results_df
    .sort_values(
        "F1 Score",
        ascending=False
    )
    .reset_index(drop=True)
)

display(
    classification_results_df
)
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	XGBoost	0.967764	0.964737	0.967764	0.963404	0.997712
1	Random Forest	0.925718	0.934165	0.925718	0.929508	0.987825
2	Logistic Regression	0.832572	0.939971	0.832572	0.873040	0.975571

Next steps: Generate code with `classification_results_df` New interactive sheet

Classification Comparison

```
best_classification_name = (
    classification_results_df
    .iloc[0]["Model"]
)

best_classification_model = (
    classification_pipelines[
        best_classification_name
    ]
)

print(
    "🏆 Best Classification Model:"
)

print(
    best_classification_name
)
```

```
🏆 Best Classification Model:
XGBoost
```

Best Classification Model

```
class_predictions = best_classification_model.predict(X_test)
print(
    classification_report(
        y_test_class,
        class_predictions,
        target_names=label_encoder.classes_,
        zero_division=0
    )
)
```

```
                precision    recall  f1-score   support

    Eligible       0.94      0.98      0.96     11100
   High_Risk       0.82      0.42      0.56      2623
Not_Eligible       0.98      1.00      0.99     46924

    accuracy                           0.97     60647
   macro avg       0.91      0.80      0.83     60647
weighted avg       0.96      0.97      0.96     60647
```

Confusion Matrix

```
class_predictions = (
    best_classification_model
    .predict(X_test)
)

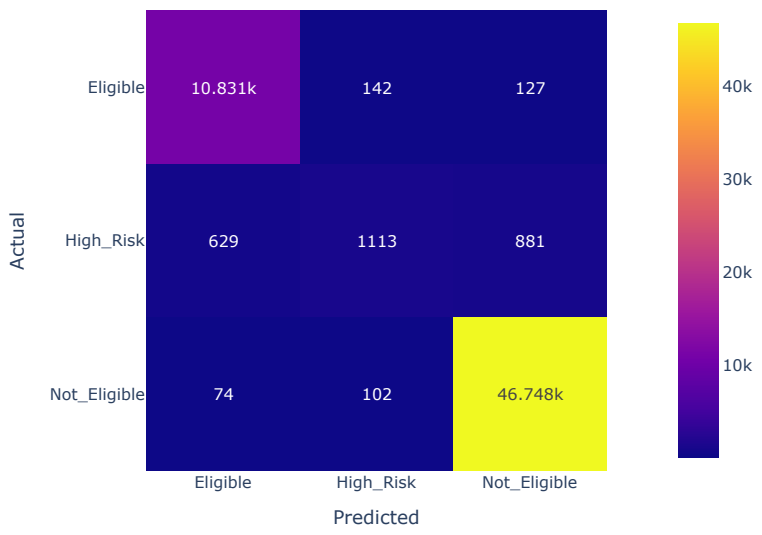
cm = confusion_matrix(
    y_test_class,
    class_predictions
)

fig = px.imshow(
    cm,
    x=label_encoder.classes_,
    y=label_encoder.classes_,
    text_auto=True,
    title=(
        "Confusion Matrix - "
        + best_classification_name
    )
)

fig.update_xaxes(
    title="Predicted"
)

fig.update_yaxes(
    title="Actual"
)

fig.show()
```



Classification Report

```
print(
    classification_report(
        y_test_class,
        class_predictions,
        target_names=label_encoder.classes_,
        zero_division=0
    )
)
```

```
              precision    recall  f1-score   support

    Eligible       0.94      0.98      0.96     11100
   High_Risk       0.82      0.42      0.56      2623
Not_Eligible       0.98      1.00      0.99     46924

    accuracy                           0.97     60647
   macro avg       0.91      0.80      0.83     60647
weighted avg       0.96      0.97      0.96     60647
```

Regression Models

```
regression_models = {

    "Linear Regression":
        LinearRegression(),

    "Random Forest Regressor":
        RandomForestRegressor(
            n_estimators=120,
            max_depth=18,
            min_samples_split=5,
            random_state=42,
            n_jobs=-1
        ),

    "XGBoost Regressor":
        XGBRegressor(
            n_estimators=180,
            max_depth=8,
            learning_rate=0.08,
            subsample=0.8,
            colsample_bytree=0.8,
            objective="reg:squarederror",
            random_state=42,
            n_jobs=-1
        )
}
```

Train Regression Models

```
mlflow.set_experiment(
    "EMIPredict_AI_Regression"
)

regression_results = []

regression_pipelines = {}

for name, model in regression_models.items():

    print("\n" + "=" * 60)
    print("Training:", name)
    print("=" * 60)

    pipeline = Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            model
        )
```

```
    ])

    with mlflow.start_run(
        run_name=name.replace(" ", "_")
    ):

        pipeline.fit(
            X_train,
            y_train_reg
        )

        predictions = pipeline.predict(
            X_test
        )

        rmse = np.sqrt(
            mean_squared_error(
                y_test_reg,
                predictions
            )
        )

        mae = mean_absolute_error(
            y_test_reg,
            predictions
        )

        r2 = r2_score(
            y_test_reg,
            predictions
        )

        denominator = np.where(
            np.abs(y_test_reg) < 1e-9,
            1,
            np.abs(y_test_reg)
        )

        mape = np.mean(
            np.abs(
                (y_test_reg - predictions)
                / denominator
            )
        ) * 100

        mlflow.log_param(
            "model",
            name
        )

        mlflow.log_metric(
            "RMSE",
            rmse
        )

        mlflow.log_metric(
            "MAE",
            mae
        )

        mlflow.log_metric(
            "R2",
            r2
        )

        mlflow.log_metric(
            "MAPE",
            mape
        )

        mlflow.sklearn.log_model(
            pipeline,
            name="model",
            skops_trusted_types=['numpy.dtype', 'sklearn.compose._column_transformer._RemainderColsList', 'xgboost
        )

        regression_pipelines[
            name
```

```
        ] = pipeline

        regression_results.append({

            "Model": name,

            "RMSE": rmse,

            "MAE": mae,

            "R2": r2,

            "MAPE": mape

        })

        print(
            "RMSE:",
            round(rmse, 2)
        )

        print(
            "MAE:",
            round(mae, 2)
        )

        print(
            "R2:",
            round(r2, 4)
        )
```

```
============================================================
Training: Linear Regression
============================================================
RMSE: 2878.86
MAE: 2047.84
R2: 0.8546

============================================================
Training: Random Forest Regressor
============================================================
RMSE: 1016.93
MAE: 260.21
R2: 0.9819

============================================================
Training: XGBoost Regressor
============================================================
RMSE: 617.26
MAE: 190.69
R2: 0.9933
```

Regression Comparison

```
regression_results_df = pd.DataFrame(
    regression_results
)

regression_results_df = (
    regression_results_df
    .sort_values(
        "RMSE",
        ascending=True
    )
    .reset_index(drop=True)
)

display(
    regression_results_df
)
```

	Model	RMSE	MAE	R2	MAPE
0	XGBoost Regressor	617.261556	190.689566	0.993317	7.028152
1	Random Forest Regressor	1016.928275	260.205008	0.981861	8.652503
2	Linear Regression	2878.860018	2047.843398	0.854630	151.062131

Next steps: Generate code with regression_results_df | New interactive sheet

Best Regression Model

```
best_regression_name = (
    regression_results_df
    .iloc[0]["Model"]
)

best_regression_model = (
    regression_pipelines[
        best_regression_name
    ]
)

print(
    "🏆 Best Regression Model:"
)

print(
    best_regression_name
)
```

```
🏆 Best Regression Model:
XGBoost Regressor
```

Actual vs Predicted

```
regression_predictions = (
    best_regression_model
    .predict(X_test)
)

comparison_df = pd.DataFrame({

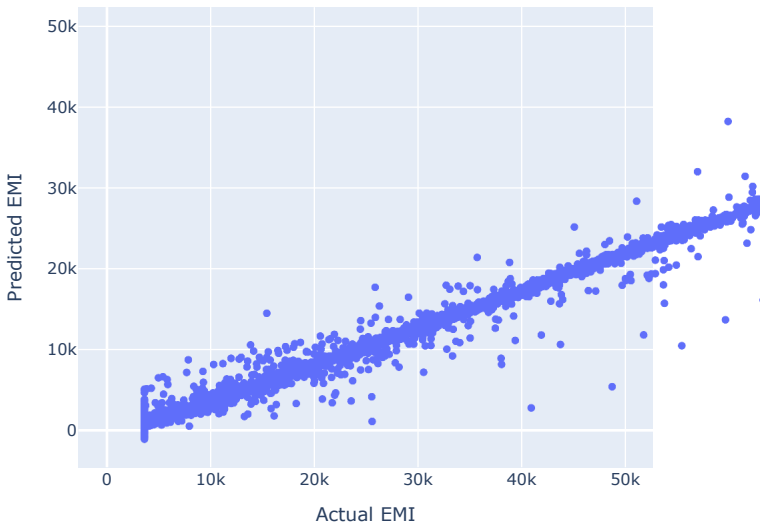
    "Actual EMI":
        y_test_reg.values,

    "Predicted EMI":
        regression_predictions

})

fig = px.scatter(
    comparison_df.sample(
        min(
            15000,
            len(comparison_df)
        ),
        random_state=42
    ),
    x="Actual EMI",
    y="Predicted EMI",
    trendline="ols",
    title=(
        "Actual vs Predicted Maximum EMI"
    )
)

fig.show()
```



Save Models

```
MODEL_DIR = (
    "/content/EMIPredict_AI/models"
)

REPORT_DIR = (
    "/content/EMIPredict_AI/reports"
)

os.makedirs(
    MODEL_DIR,
    exist_ok=True
)

os.makedirs(
    REPORT_DIR,
    exist_ok=True
)

joblib.dump(
    best_classification_model,
    os.path.join(
        MODEL_DIR,
        "best_classification_model.pkl"
    )
)

joblib.dump(
    best_regression_model,
    os.path.join(
        MODEL_DIR,
        "best_regression_model.pkl"
    )
)

joblib.dump(
    label_encoder,
    os.path.join(
        MODEL_DIR,
        "label_encoder.pkl"
    )
)

classification_results_df.to_csv(
    os.path.join(
        REPORT_DIR,
        "classification_results.csv"
```

```
    ),
    index=False
)

regression_results_df.to_csv(
    os.path.join(
        REPORT_DIR,
        "regression_results.csv"
    ),
    index=False
)

print("✅ Models and reports saved.")
```

```
✅ Models and reports saved.
```

Risk Score

```
def calculate_risk_score(
    credit_score,
    monthly_salary,
    current_emi,
    total_expenses,
    emergency_fund
):

    salary = max(
        float(monthly_salary),
        1
    )

    debt_ratio = (
        current_emi / salary
    )

    expense_ratio = (
        total_expenses / salary
    )

    score = 100

    # Credit score
    if credit_score < 550:
        score -= 35

    elif credit_score < 650:
        score -= 20

    elif credit_score < 750:
        score -= 10

    # EMI burden
    if debt_ratio > 0.50:
        score -= 30

    elif debt_ratio > 0.40:
        score -= 20

    elif debt_ratio > 0.30:
        score -= 10

    # Expense burden
    if expense_ratio > 0.70:
        score -= 25

    elif expense_ratio > 0.50:
        score -= 15

    # Emergency savings
    if emergency_fund < salary:
        score -= 10

    score = max(
        0,
        min(100, score)
    )
```

```
    if score >= 75:
        risk_level = "Low Risk"

    elif score >= 50:
        risk_level = "Medium Risk"

    else:
        risk_level = "High Risk"

    return score, risk_level
```

Gradio Prediction Function

```
def predict_emi(
    age,
    gender,
    marital_status,
    education,
    monthly_salary,
    employment_type,
    years_of_employment,
    company_type,
    house_type,
    monthly_rent,
    family_size,
    dependents,
    school_fees,
    college_fees,
    travel_expenses,
    groceries_utilities,
    other_monthly_expenses,
    existing_loans,
    current_emi_amount,
    credit_score,
    bank_balance,
    emergency_fund,
    emi_scenario,
    requested_amount,
    requested_tenure
):

    input_data = pd.DataFrame([{

        "age": age,

        "gender": gender,

        "marital_status":
            marital_status,

        "education":
            education,

        "monthly_salary":
            monthly_salary,

        "employment_type":
            employment_type,

        "years_of_employment":
            years_of_employment,

        "company_type":
            company_type,

        "house_type":
            house_type,

        "monthly_rent":
            monthly_rent,

        "family_size":
            family_size,
```

```
        "dependents":
            dependents,

        "school_fees":
            school_fees,

        "college_fees":
            college_fees,

        "travel_expenses":
            travel_expenses,

        "groceries_utilities":
            groceries_utilities,

        "other_monthly_expenses":
            other_monthly_expenses,

        "existing_loans":
            existing_loans,

        "current_emi_amount":
            current_emi_amount,

        "credit_score":
            credit_score,

        "bank_balance":
            bank_balance,

        "emergency_fund":
            emergency_fund,

        "emi_scenario":
            emi_scenario,

        "requested_amount":
            requested_amount,

        "requested_tenure":
            requested_tenure

    }])

    # SAME feature engineering as training
    input_engineered = (
        feature_engineering(
            input_data
        )
    )

    # Classification
    class_prediction = (
        best_classification_model
        .predict(
            input_engineered
        )[0]
    )

    class_label = (
        label_encoder
        .inverse_transform(
            [class_prediction]
        )[0]
    )

    # Regression
    predicted_max_emi = (
        best_regression_model
        .predict(
            input_engineered
        )[0]
    )

    predicted_max_emi = max(
        0,
        float(predicted_max_emi)
    )
```

```
    # Total expenses
    total_expenses = (
        monthly_rent
        + school_fees
        + college_fees
        + travel_expenses
        + groceries_utilities
        + other_monthly_expenses
    )

    # Risk score
    risk_score, risk_level = (
        calculate_risk_score(
            credit_score,
            monthly_salary,
            current_emi_amount,
            total_expenses,
            emergency_fund
        )
    )

    # Recommendation
    if class_label == "Eligible":

        recommendation = (
            "The customer appears financially "
            "suitable for the requested EMI."
        )

    elif class_label == "High_Risk":

        recommendation = (
            "This is a marginal financial case. "
            "Consider a lower loan amount, longer "
            "tenure, or additional verification."
        )

    else:

        recommendation = (
            "The current financial profile is not "
            "recommended for the requested loan."
        )

    result = f"""
# 💰 EMIPredict AI

## EMI Eligibility
### {class_label}

---

## Maximum Safe Monthly EMI

### ₹{predicted_max_emi:,.2f}

---

## Financial Risk Score

### {risk_score}/100

**Risk Level:** {risk_level}

---

## Requested Loan

₹{requested_amount:,.2f}

## Requested Tenure

{requested_tenure} months

## Credit Score

{credit_score}
```

```
---

## Models Used

**Classification:** {best_classification_name}

**Regression:** {best_regression_name}

---

## Recommendation

{recommendation}

> This result is an ML-based financial assessment and
> should not be treated as a final lending decision.
"""

    return result
```

Professional Gradio UI

```
with gr.Blocks(
    title="EMIPredict AI"
) as demo:

    gr.Markdown(
        """
        # 💰 EMIPredict AI
        ## Intelligent Financial Risk Assessment Platform

        **AI-powered EMI Eligibility + Maximum Safe EMI Prediction**
        """
    )

    with gr.Tab(
        "🏦 Financial Risk Assessment"
    ):

        with gr.Row():

            with gr.Column():

                age = gr.Number(
                    value=30,
                    label="Age"
                )

                gender = gr.Dropdown(
                    [
                        "Male",
                        "Female"
                    ],
                    value="Male",
                    label="Gender"
                )

                marital_status = gr.Dropdown(
                    [
                        "Single",
                        "Married"
                    ],
                    value="Single",
                    label="Marital Status"
                )

                education = gr.Dropdown(
                    [
                        "High School",
                        "Graduate",
                        "Post Graduate",
                        "Professional"
                    ],
                    value="Graduate",
                    label="Education"
```

```
            )

            monthly_salary = gr.Number(
                value=60000,
                label="Monthly Salary (₹)"
            )

            employment_type = gr.Dropdown(
                [
                    "Private",
                    "Government",
                    "Self-employed"
                ],
                value="Private",
                label="Employment Type"
            )

            years_of_employment = gr.Number(
                value=5,
                label="Years of Employment"
            )

            company_type = gr.Dropdown(
                [
                    "Small",
                    "Medium",
                    "Large",
                    "MNC"
                ],
                value="Large",
                label="Company Type"
            )

            house_type = gr.Dropdown(
                [
                    "Rented",
                    "Own",
                    "Family"
                ],
                value="Rented",
                label="House Type"
            )

        with gr.Column():

            monthly_rent = gr.Number(
                value=10000,
                label="Monthly Rent (₹)"
            )

            family_size = gr.Number(
                value=4,
                label="Family Size"
            )

            dependents = gr.Number(
                value=2,
                label="Dependents"
            )

            school_fees = gr.Number(
                value=5000,
                label="School Fees (₹)"
            )

            college_fees = gr.Number(
                value=0,
                label="College Fees (₹)"
            )

            travel_expenses = gr.Number(
                value=3000,
                label="Travel Expenses (₹)"
            )

            groceries_utilities = gr.Number(
                value=8000,
                label="Groceries & Utilities (₹)"
            )
```

```
            )

            other_monthly_expenses = gr.Number(
                value=3000,
                label="Other Monthly Expenses (₹)"
            )

            existing_loans = gr.Dropdown(
                [
                    "Yes",
                    "No"
                ],
                value="No",
                label="Existing Loans"
            )

        with gr.Column():

            current_emi_amount = gr.Number(
                value=5000,
                label="Current EMI (₹)"
            )

            credit_score = gr.Number(
                value=720,
                label="Credit Score"
            )

            bank_balance = gr.Number(
                value=100000,
                label="Bank Balance (₹)"
            )

            emergency_fund = gr.Number(
                value=150000,
                label="Emergency Fund (₹)"
            )

            emi_scenario = gr.Dropdown(
                [
                    "E-commerce Shopping EMI",
                    "Home Appliances EMI",
                    "Vehicle EMI",
                    "Personal Loan EMI",
                    "Education EMI"
                ],
                value="Personal Loan EMI",
                label="EMI Scenario"
            )

            requested_amount = gr.Number(
                value=300000,
                label="Requested Amount (₹)"
            )

            requested_tenure = gr.Number(
                value=36,
                label="Requested Tenure (Months)"
            )

    predict_button = gr.Button(
        "🔍 Assess Financial Risk",
        variant="primary"
    )

    prediction_output = gr.Markdown()

    prediction_inputs = [
        age,
        gender,
        marital_status,
        education,
        monthly_salary,
        employment_type,
        years_of_employment,
        company_type,
        house_type,
        monthly_rent,
        family_size,
```

```
            dependents,
            school_fees,
            college_fees,
            travel_expenses,
            groceries_utilities,
            other_monthly_expenses,
            existing_loans,
            current_emi_amount,
            credit_score,
            bank_balance,
            emergency_fund,
            emi_scenario,
            requested_amount,
            requested_tenure
        ]

        predict_button.click(
            fn=predict_emi,
            inputs=prediction_inputs,
            outputs=prediction_output
        )

    with gr.Tab(
        "📊 Model Performance"
    ):

        gr.Markdown(
            "## Classification Models"
        )

        gr.Dataframe(
            classification_results_df
        )

        gr.Markdown(
            "## Regression Models"
        )

        gr.Dataframe(
            regression_results_df
        )

        gr.Markdown(
            f"""
            ### 🏆 Best Models

            **Classification:** {best_classification_name}

            **Regression:** {best_regression_name}
            """
        )

    with gr.Tab(
        "📁 Dataset Explorer"
    ):

        gr.Markdown(
            f"""
            ### Dataset Information
```

EMIPredict AI

Intelligent Financial Risk Assessment Platform

AI-powered EMI Eligibility + Maximum Safe EMI Prediction

Financial Risk Assessment | Model Performance | Dataset Explorer | Ab

Age

30

Gender

Male

Marital Status

Single

Education

Graduate

Monthly Salary (₹)

60000

Employment Type

Private

Years of Employment

5

Company Type

Large

House Type

Rented

Monthly Rent (₹)

10000

Family Size

4

Dependents

2

School Fees (₹)

5000

College Fees (₹)

0

Travel Expenses (₹)

3000

Groceries & Utilities (₹)

8000

Other Monthly Expenses (₹)

3000

Existing Loans

No

Current EMI (₹)

5000

Credit Score

720

Bank Balance (₹)

100000

Emergency Fund (₹)

150000

EMI Scenario

Personal Loan EMI

Requested Amount (₹)

300000

Requested Tenure (Months)

36

🔍 Assess Financial Risk

💰 EMIPredict AI

EMI Eligibility

Not_Eligible

Maximum Safe Monthly EMI

₹6,789.45

Financial Risk Score

90/100

Risk Level: Low Risk

Requested Loan

₹300,000.00

Requested Tenure

36 months

Credit Score

720

Models Used

Classification: XGBoost

Regression: XGBoost Regressor

Recommendation

The current financial profile is not recommended for the requested loan.

> This result is an ML-based financial assessment and should not be treated as a final lending decision.